

# EXPERIMENT 11

240701426

## A. First-Come,First-Served CPU Scheduling Algorithm

**FACE Prep**

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Save

Q1

Completed

### First-Come,First-Served CPU Scheduling Algorithm

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

**Input Format**

1. The first input line contains an integer representing the number of processes.
2. The next lines contain two integers for each process, where the first integer represents the Arrival Time (AT) and the second integer

Select Language : C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main(){
4     int n,i;
5     int at[20],bt[20],ct[20],tat[20],wt[20];
6     float avg_tat = 0 ,avg_wt = 0;
7     printf("Enter number of processes: \n");
8     scanf("%d",&n);
9
10    for(i=0;i<n;i++){
11        printf("Enter Arrival Time and Burst Time for P%d: \n",i+1);
12        scanf("%d %d", &at[i],&bt[i]);
13    }
14
15    for (i =0;i<n;i++){
16        if(i==0){
17            ct[i]=at[i]+bt[i];
18        }else{
19            if(ct[i-1]<at[i])
20                ct[i] = at[i] + bt[i];
21            else
22                ct[i] = ct[i-1] + bt[i];
23        }
24    }
25    for(i=0;i<n;i++){
26        tat[i] = ct[i] - at[i];
27        wt[i] = tat[i] - bt[i];
```

Test Cases 2

Run Sample Cases

Run All Cases

## B. LRU Page Replacement Algorithm

**FACE Prep**

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Save

Q1

Completed

### LRU Page Replacement Algorithm

Write a C program to implement the LRU Page Replacement Algorithm. The program should accept a reference string and the number of frames. For each page request, identify whether it results in a Hit or a Fault. If a Fault occurs and the frames are full, replace the page that was accessed least recently. Display the frame status at each step and calculate the total number of Page Faults and Hits

**Input Format**

1. The first line of input contains an integer representing the number of pages in the reference string.
2. The second line contains the page reference string as a sequence of integers.
3. The third line contains an integer representing the number of available frames.

**Output Format**

The output displays the total number of Page Faults and the total number of Page Hits after processing the entire reference string using the LRU page replacement algorithm.

Select Language : C (GCC 9.2.0)

```
1 #include<stdio.h>
2
3 int main(){
4     int n,f;
5
6     printf("Enter number of pages in reference string: \n");
7     scanf("%d",&n);
8
9     int ref[n];
10    printf("Enter the reference string: \n");
11    for(int i=0;i<n;i++)
12        scanf("%d",&ref[i]);
13
14    printf("Enter number of frames: \n");
15    scanf("%d",&f);
16
17    int frame[f],time[f];
18    int faults =0,hits=0;
19    int counter=0;
20
21    for(int i=0;i<f;i++){
22        frame[i]=-1;
23        time[i]=0;
24    }
25    for(int i=0;i<n;i++){
26        int found =0;
```

Test Cases 2

Run Sample Cases

Run All Cases